[UNIVERSITY NAME]

Department of Computer Science & Engineering

AI SEARCH ALGORITHM VISUALIZER

AN INTERACTIVE GRID-BASED SYSTEM FOR CLASSICAL AI SEARCH ALGORITHMS

A Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering.

Submitted By:

Name: Shafayet Hossain

ID: [Student Roll/ID]

Semester: [Spring / Fall Year]

Batch: [Batch No.]

Supervised By:

Name: [Supervisor Name]

Designation: [Assistant Professor / Lecturer]

Department of Computer Science & Engineering

[University Name]

Date of Submission: [Submission Date]

1. Abstract

This project presents an interactive visualization system for understanding classical artificial intelligence search algorithms in a grid-based environment. The system allows users to create walls, assign terrain weights, select start and goal states, and observe how algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), A* Search, and Greedy Best-First Search explore the search space and determine a path.

The primary objective is to transform abstract search concepts into an observable and educational experience. Instead of displaying only the final route, the application visualizes node expansion, frontier management, heuristic evaluation, path reconstruction, and cost-based decision-making. This makes it easier to compare uninformed and informed search strategies under different obstacle and terrain conditions.

The report discusses the theoretical foundations, system architecture, implementation approach, functional workflow, evaluation metrics, observed algorithmic behavior, limitations, and possible future improvements.

2. Introduction

Artificial Intelligence plays a central role in robotics, autonomous navigation, planning, optimization, and decision-making. One of the most fundamental AI problems is pathfinding, where an agent searches for a valid route from a starting state to a target state while avoiding obstacles and, in many cases, minimizing travel cost.

Classical search algorithms provide different strategies for exploring a state space. Some algorithms explore without domain knowledge, while others use heuristic information to guide the search toward a goal. Their behavior can vary significantly depending on the map structure, obstacles, and movement costs.

This project develops a graphical visualizer that demonstrates these differences on a two-dimensional grid. Users can edit the environment and observe the search process through animation, making the application suitable for AI laboratory demonstrations, classroom learning, and algorithm comparison.

2.1 Project Objectives

- Implement classical AI search algorithms in Python.
- Visualize the exploration process step by step on a 2D grid.
- Allow users to create walls, erase cells, assign weighted terrain, and place start and goal states.
- Compare algorithm behavior using explored nodes, path length, path cost, and execution time.
- Provide an educational graphical interface for AI laboratory practice.
- Demonstrate the difference between uninformed, cost-sensitive, and heuristic-guided search.

3. Literature Review and Background

Search algorithms operate over a state space or graph, where nodes represent possible states and edges represent transitions. In grid-based pathfinding, each cell is treated as a node, and valid movement between neighboring cells forms the edges of the graph.

3.1 Uninformed Search

- **Breadth-First Search (BFS):** Explores nodes level by level using a queue. It guarantees the shortest path in an unweighted graph when all moves have equal cost.
- **Depth-First Search (DFS):** Explores one branch deeply before backtracking. It can find a route but does not guarantee the shortest or least-cost path.
- **Uniform Cost Search (UCS):** Expands the node with the lowest cumulative path cost. It is appropriate for weighted environments.

3.2 Informed Search

- **A* Search:** Combines the cost already incurred and an estimated remaining cost using f(n) = g(n) + h(n).
- **Greedy Best-First Search:** Uses only the heuristic value h(n) to prioritize nodes that appear closer to the goal.

3.3 Heuristic Function

For four-directional grid movement, Manhattan distance is a suitable heuristic because diagonal movement is not allowed. It is calculated as:

$$h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

4. System Design

The application is designed as a Python-based graphical system composed of a user interface, grid model, search engine, metric calculator, and documentation module.

4.1 Main Components

- **main.py:** User interface, grid controls, animation, and event handling.
- **grid.py:** Grid representation, cell properties, walls, weights, and neighbor validation.
- **algorithms.py:** Implementations of BFS, DFS, UCS, A*, and Greedy Best-First Search.
- **metrices.py:** Calculation and presentation of performance metrics.
- **README.md:** Installation, usage instructions, and project documentation.

4.2 Grid Model

The environment is represented by a two-dimensional grid. Each cell stores whether it is blocked, its movement cost, and whether it represents the start or goal state. The grid supports wall placement, cell clearing, weighted terrain assignment, neighbor checking, and random maze generation.

4.3 Graph Representation

Every traversable grid cell corresponds to a graph node. Edges connect valid neighboring cells. A normal cell may have a default movement cost, while weighted cells impose additional traversal cost. This representation allows the same environment to be used for both unweighted and weighted search algorithms.

5. Implementation

5.1 GUI and Interaction

The system uses Tkinter, Python’s standard GUI toolkit, to provide an interactive interface. The interface includes algorithm selection, wall creation, erasing, weight assignment, start and goal placement,

animation speed control, live status information, pause/resume support, random maze generation, and a comparison window.

5.2 Search Algorithms Implemented

BFS: Uses a queue to explore neighboring cells in layers. It is effective for shortest-path discovery when all movement costs are equal.

DFS: Uses a stack or recursive-style traversal to explore deeply before backtracking. It may discover a route quickly but can produce a long path.

UCS: Uses a priority queue to expand the cell with the lowest accumulated cost. It considers terrain weights and aims to find the least-cost route.

A*: Uses f(n) = g(n) + h(n), where g(n) is the cost from the start and h(n) estimates the remaining cost to the goal. With a suitable heuristic, it can efficiently find an optimal path.

Greedy Best-First Search: Prioritizes nodes using only h(n). It often moves toward the goal quickly but may ignore expensive terrain or produce a suboptimal route.

5.3 Path Reconstruction

After the goal is reached, the system reconstructs the final path by following parent pointers from the goal back to the start. The reconstructed sequence is then reversed and displayed on the grid.

5.4 Metrics Calculation

- Number of nodes explored.
- Final path length.
- Total path cost.
- Execution time.

These metrics are displayed in the analytics panel and support side-by-side algorithm comparison.

6. Functional Workflow

The application follows the workflow below:

- The user selects an algorithm.
- The user configures the grid by adding walls, weights, and start/goal states.
- The user starts the search.
- The selected algorithm explores the grid.
- The interface animates visited nodes and frontier activity.
- The system reconstructs and displays the final path.
- Performance metrics are calculated and shown.
- The user may repeat the experiment or compare algorithms on the same map.

7. Results and Discussion

The visualizer demonstrates important behavioral differences among the implemented algorithms. The following observations describe the expected behavior when the algorithms are executed on equivalent grid environments.

7.1 BFS

BFS explores the search space layer by layer and generally finds the shortest path in an unweighted grid. However, it may explore many nodes because it does not use information about the direction of the goal.

7.2 DFS

DFS can reach the goal without exploring the entire grid, but its route depends strongly on neighbor ordering. It may travel through unnecessary branches and does not guarantee minimum path length or cost.

7.3 UCS

UCS is effective in weighted environments because it expands nodes according to cumulative cost. It can avoid physically short but expensive routes and is suitable when terrain costs differ.

7.4 A*

A* balances the actual cost from the start with the estimated remaining distance. It often explores fewer irrelevant nodes than uninformed search while retaining optimality when the heuristic is appropriate.

7.5 Greedy Best-First Search

Greedy search tends to move toward the goal rapidly because it considers only the heuristic. However, it can be misled by walls or high-cost terrain and may return a longer or more expensive route.

The visualization makes these differences clear by showing not only the final path but also the order and region in which nodes are explored.

8. Advantages and Limitations

8.1 Advantages

- Effective for teaching classical AI search concepts.
- Interactive and visually intuitive.
- Built with Python and Tkinter, making it accessible for academic use.
- Supports obstacles and weighted terrain.
- Provides live statistics and algorithm comparison.
- Useful for demonstrations, experimentation, and laboratory assignments.

8.2 Limitations

- Movement is restricted to four directions; diagonal movement is not implemented.
- The environment is limited to a two-dimensional grid.
- Manhattan distance is not suitable for every possible movement model.
- The system focuses on classical search rather than advanced planning or real-world map routing.
- Performance results depend on the selected grid configuration and hardware.

9. Future Work

- Implement diagonal movement with an octile-distance heuristic.
- Add predefined benchmark maps and repeatable test scenarios.

- Support dynamic obstacles that appear during execution.
- Provide single-step execution and algorithm pause points.
- Add graphical charts for comparing explored nodes, cost, and execution time.
- Export generated mazes, routes, and performance reports.
- Extend the system to support larger graphs and real-world map data.
- Add additional algorithms such as Bidirectional Search, IDDFS, and Dijkstra's Algorithm.

10. Conclusion

The AI Search Algorithm Visualizer is a practical educational application that combines classical artificial intelligence theory with interactive graphical visualization. It implements BFS, DFS, UCS, A*, and Greedy Best-First Search in a grid-based environment containing obstacles and weighted cells.

The project demonstrates how different search strategies expand nodes, evaluate candidate states, reconstruct paths, and respond to cost or heuristic information. By allowing users to observe the complete exploration process, the system helps bridge the gap between theoretical algorithm descriptions and practical understanding.

Overall, the project achieves its objective of making AI search algorithms easier to understand, compare, and analyze. It can serve as a useful tool for AI laboratory work, classroom demonstrations, and future extensions into more advanced pathfinding and intelligent navigation systems.

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson.

2. Poole, D., & Mackworth, A. (2017). *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press.

3. Nilsson, N. J. (1980). *Principles of Artificial Intelligence*. Morgan Kaufmann.

4. Python Software Foundation. *Tkinter Documentation*.

5. Python Software Foundation. *Python Documentation: collections.deque and heapq*.

_____

Student Signature

_____

Supervisor Signature & Date